

Does the Model Check the Right Thing?

Locating verification failure in disrupted chain-of-thought reasoning — Final Research Proposal

Design Project (SWE 4606 / CSE 4610) · Systems and Software Lab · Department of
CSE, Islamic University of Technology

12 September 2026

Contents

1. Executive summary	3
2. Motivation	3
2.1 The problem with outcome-only evaluation	3
2.2 Why targeting, specifically	4
2.3 Why this matters beyond the laboratory	4
3. Definitions	4
4. Related work	5
4.1 Controlled disruption of chain-of-thought	5
4.2 Graphs as a post-hoc diagnostic	6
4.3 Error localization	6
4.4 The gap	7
5. Research question	7
6. The verification funnel	7
7. Hypotheses	8
H1 — Targeting differs by fault type. (<i>Primary</i>)	8
H2 — Structure sees what surface cues cannot. (<i>Primary</i>)	8
H3 — Source framing matters independently of content. (<i>Secondary</i>)	9
H4 — The instrument must be validated on the text it is applied to. (<i>Methodological</i>)	9
8. Representation	9
8.1 Formal pipeline	9
8.2 Nodes: the three-way scheme	9
8.3 Edges	10
8.4 Target categories	10
8.5 Node matching across graphs	11

9. Perturbation taxonomy	11
9.1 Generation mechanism	11
9.2 The four faults plus control	11
9.3 Contamination set — defined before analysis	12
9.4 The skipped-step condition needs a ghost node	12
10. Metrics	12
11. Experiments	13
Ablations	14
12. Statistical framework	14
13. Validation protocol and the gate	14
13.1 Annotation set	15
13.2 Thresholds	15
13.3 The gate	15
14. Expected outcomes	16
15. Contributions	16
16. Limitations	17
17. Threats to validity and their controls	17
18. Scope: this semester versus the full thesis	18
Achievable this semester	18
Explicitly deferred to the thesis	18
Prototype deliverable	18
19. Budget	19
20. Final thesis statement	19
21. References	20
Priority reading list	21

1. Executive summary

When a language model writes out its reasoning step by step and that reasoning is disturbed, the research field currently records one of three things: whether the final answer changed, whether the model recovered the right answer, or how much doubt-expressing language appeared afterwards. All three are properties of the text taken as a whole.

None of them can answer a more specific question: when the model pauses to check its work, **which earlier step is it actually checking?**

That question cannot be asked of a flat list of sentences, because it concerns a relationship between two steps rather than a property of one. It can be asked of a graph, where that relationship is stored explicitly as a link.

This project converts each chain of reasoning into a small graph, deliberately damages one step in four controlled ways, and then measures whether the model’s own spontaneous checking is structurally directed at the damage. Because we plant the damage ourselves, we know its exact location on every problem at no cost — which lets us score the model’s checking the way one scores a search: did it look in the right place, and if not, how far off was it.

The project separates four behaviours that existing work collapses into “self-correction”: whether the model checks at all, whether the check is aimed correctly, whether it detects the fault, and whether it repairs the answer. We call this the **verification funnel**, and measuring it is the central contribution.

A second contribution is methodological. Every comparable graph-parsing pipeline we surveyed validates its labelling against human annotators on clean text and then applies the same labeller to deliberately corrupted text without re-checking it. We measure labelling agreement separately on damaged and undamaged text, report the difference as a result in its own right, and treat a significant drop as grounds for withholding automated structural claims.

Total estimated cost: under US\$60. No GPU or frontier-model access required.

2. Motivation

2.1 The problem with outcome-only evaluation

The standard experimental design for studying disrupted reasoning is:

Question → generate reasoning → inject a fault → generate the remainder → measure whether the answer changed.

This produces one number per (model, fault type) pair. That number cannot distinguish between a model that:

1. never noticed the fault and computed straight through it;
2. noticed something was wrong, checked, but checked the wrong step;
3. checked the right step, identified the fault, and was overruled anyway;
4. had no checking behaviour to begin with.

All four produce “wrong answer.” All four are different failures requiring different interventions. A model that never checks needs a different fix from one that checks the wrong place, which needs a different fix again from one that checks correctly and then ignores what it found.

2.2 Why targeting, specifically

Recent work has begun to measure checking behaviour rather than only outcomes — for example by counting doubt-expressing sentences in a fixed window after an injection. This is a real advance, and it establishes that models do respond to disturbance. But it measures the *volume* of checking, not its *direction*.

Consider a concrete case. We insert a false claim attributed to a professor. The model then writes:

“Let me verify this carefully. Checking the second multiplication: $25 \times 4 = 100$. That is correct. So the total is $150 + 100 = 250$.”

Every measurement in the current literature scores this as verification having occurred. The model used checking language; a doubt classifier fires; a keyword count registers a hit. But the check examined a step that was never damaged, and the professor’s false claim passed through untouched.

We call this **verification misdirection**. It is invisible to any method that treats a check as a property of the sentence containing it, and visible immediately to any method that records which earlier step the check refers to.

2.3 Why this matters beyond the laboratory

The four fault types map onto real deployment conditions in which a model consumes reasoning it did not produce:

Fault type	Real-world analogue
Arithmetic error	A tool call, spreadsheet, or upstream agent returns a wrong number
Authority statement	A retrieved document or user message asserts something false with apparent credibility
Skipped step	A prior reasoning trace omits a necessary justification
Contradictory step	Two retrieved sources or two agents disagree

In multi-agent systems, retrieval-augmented generation, automated data analysis, and tutoring systems, a model must constantly decide whether to trust, check, revise, or continue from supplied material. Knowing whether its visible checking behaviour is a reliable signal of actual auditing is directly relevant to whether that behaviour can be used for oversight.

3. Definitions

The whole proposal uses only these terms. Each is defined once, here.

Chain-of-thought (CoT). Intermediate reasoning a model writes out before its final answer, rather than answering immediately.

Reasoning step. One numbered line or sentence of that reasoning. Example: “The first worker costs $\$20 \times 5 = \100 .”

Verification step. A reasoning step whose function is to examine an *earlier* step rather than to produce new content. Example: “Let me double-check: $\$100 + \$100 = \$200$.”

Perturbation (injection). A deliberate, controlled fault we insert into the reasoning.

Prefix-conditioned continuation. The generation method: we supply the model with the first few steps of its own solution, one of them damaged, and ask it to continue from the

next step. This studies mid-stream recovery rather than after-the-fact critique of a finished document.

Reasoning graph. A directed graph in which each step is a node, and two kinds of arrow are recorded: sequence (step i to step $i+1$) and back-reference (a later step to the specific earlier step it examines).

Back-reference edge. The link from a verification step to the earlier step it is about. This is the object the whole project is built on.

Injected node. The node containing the fault we planted. Known exactly, by construction.

Contamination set. The injected node plus the pre-specified set of later nodes whose content depends on it. Defined before analysis begins (Section 9.3).

Audit. A back-reference edge from a verification step landing inside the contamination set.

4. Related work

Nine papers bound this proposal. They are grouped by what they contribute and what they leave open. Full citations are in Section 18.

4.1 Controlled disruption of chain-of-thought

Lanham et al. (2023) established the design: deliberately add mistakes to a chain of thought, then measure the consistency between the altered reasoning and the final answer. The readout is answer-level.

Aravindan and Kejriwal (2026), “Fragile Thoughts” is the closest work to ours on experimental setup. They evaluate thirteen models on a taxonomy of five perturbation types — MathError, UnitConversion, Sycophancy, SkippedSteps, ExtraSteps — injected into reasoning chains on mathematical tasks, using GSM8K and a partial-trace completion design in which the model receives a question plus the first few solution steps with perturbations injected at specific points and must generate the remainder.

Three of their five types are effectively equivalent to three of ours. Their dataset, injection mechanism, and correctness filtering match ours closely. **We must not claim novelty on the perturbation taxonomy.**

Their dependent variable is accuracy. Notably, they *observe* the phenomenon we propose to measure but have no instrument for it: they present a model that opens with a statement about carefully reviewing the partial solution for mistakes, reviews something, and never identifies the planted error — flagging it as relevant to interpretability and moving on. Our framework is the systematic measurement of that single hand-picked observation.

Two of their findings constrain our design. On GSM8K, average accuracy drops were 18% for MathError and 9% for UnitConversion but only around 3% for Sycophancy and 2% for SkippedSteps — most large models are near ceiling. Smaller models show much larger effects: Llama 3.1 8B-Instruct dropped 35% under MathError and 17% under Sycophancy. This justifies our choice of 7–8B models and warns against anything larger.

von Recum, Girrbach and Akata (2026) perturb a model’s own chain at fixed timesteps with seven interventions grouped as benign, neutral and adversarial, across nine open-weight reasoning models. Critically, they go beyond accuracy: they classify each of the twenty sentences following an intervention as expressing doubt or not, and find that doubt spikes immediately after intervention with magnitude depending on intervention type. Their doubt classifier achieved 93.75% accuracy and Cohen’s κ of 0.8742 against a majority of four human annotators.

This is the closest prior work on our dependent variable, and it changes what we may claim. We cannot say that no one has asked whether different fault types leave different traces in checking behaviour. They asked and answered. What their measure cannot do is say which earlier step the doubt is about — it is a proportion over a positional window, with no structure, no node correspondence between clean and damaged chains, and no notion of target.

Tsui (2025), “Self-Correction Bench” injects the same error under two matched conditions, one where it appears as the model’s own prior output and one where it is externally supplied, across fourteen open-source non-reasoning models, and reports an average 64.5% self-correction blind-spot rate: models correct an externally attributed error but fail on the same error presented as their own. Appending a minimal “Wait” prompt reduced the blind spot by 89.3%.

This is a strong precedent for our Hypothesis 3 and we must cite it as such. **We cannot claim novelty for “controlling the source framing of a planted error.”** That has been done. What remains open is measuring where checking is directed after such an injection. Their finding that a one-word prompt dramatically changes correction behaviour also strengthens our focus on *spontaneous* rather than elicited checking.

4.2 Graphs as a post-hoc diagnostic

Lee et al. (2025), “ReasoningFlow” builds the parser we need. Traces are segmented into nodes assigned one of eight semantic classes, connected into a labelled directed acyclic graph with three main edge types and fourteen fine-grained labels, surfacing verification, backtracking and proof-by-contradiction as subgraph patterns. Their manual validation covers 31 questions with four annotators, two per trace, reaching Krippendorff’s α of 0.8851 for node classification and 0.9193 for edge detection.

Every trace they validated is undamaged. **They never apply the parser to deliberately corrupted text and never test whether agreement holds there.** This is the gap our Hypothesis 4 occupies.

They also report that mechanistically measured causal step-to-step dependencies do not align with language-level discourse relations — a caution we absorb into Section 16: a text-parsed edge is a discourse relation, not a proven causal dependency.

Xiong et al. (2025), “Mapping the Minds of LLMs” clusters long reasoning outputs into semantically coherent steps and constructs directed reasoning graphs, finding that structural properties such as exploration density, branching and convergence ratios correlate with reasoning accuracy. Post-hoc graphs, but no planted faults and no target metric. Their pipeline relies on model-based clustering, which is more expensive than ours and reinforces our cost argument.

Besta et al. (2024), “Graph of Thoughts” and relatives use graphs as an *execution engine* — a prompting architecture that branches and iterates during generation to raise accuracy. We do not alter generation at all. This line is orthogonal and should not be confused with ours.

4.3 Error localization

Tyen et al. (2024) show that poor self-correction performance stems from models’ inability to find logical mistakes rather than their inability to correct a known mistake; supplying ground-truth mistake location in a backtracking setup boosts downstream performance across five reasoning tasks. They released BIG-Bench Mistake, a dataset of chain-of-thought outputs annotated with first-mistake location.

This is the strongest external justification for our project’s premise: **location is the bottleneck.** It also motivates our secondary oracle-location experiment (E5). Their models are *asked* to find the error; ours are not.

Zheng et al. (2024), “ProcessBench” provides 3,400 test cases, each a step-by-step solution with human-annotated error location, where models must identify the earliest erroneous step or conclude all steps are correct. Again, elicited localization on someone else’s solution. Ours is spontaneous localization inside the model’s own continuation. The contrast is clean and worth stating explicitly.

Lightman et al. (2023), “Let’s Verify Step by Step” established process supervision — rewarding correct intermediate steps rather than only correct outcomes. Background for why step-level structure matters at all.

Ling et al. (2023), “Deductive Verification of Chain-of-Thought Reasoning” proposes the Natural Program format, decomposing verification into step-by-step subprocesses each receiving only its necessary premises. Their premise-listing idea is conceptually adjacent to our back-reference edges, but it changes the generation format; we parse ordinary output after the fact.

4.4 The gap

Stated precisely, because precision is what protects us:

***Measurement gap.** Prior disruption studies have a well-designed intervention and an under-specified readout. They can say which fault types are most damaging on average, and (in the most recent work) how much checking language follows. None records which earlier step a check refers to, so none can distinguish a check that audits the fault from a check that audits something else.*

***Ground-truth gap.** Error-localization research must pay for human annotation of error positions. Controlled-injection research generates error positions for free but has never used them to score spontaneous checking behaviour. Nobody has joined the two.*

***Instrument gap.** The one family of designs that could supply a structural readout has been validated only on clean text. An unvalidated parser applied to deliberately altered input is not a measurement instrument; it is an untested confound.*

5. Research question

When a language model’s chain of reasoning is deliberately damaged mid-stream, is the model’s own spontaneous verification structurally directed at the damage — and does the pattern of direction differ by damage type?

Two words carry weight. **Spontaneous:** the model is never told a fault exists or asked to find one. **Directed:** we measure the target of checking, not its presence.

6. The verification funnel

This is the conceptual core, and it is what makes the contribution legible.

Existing work collapses four distinct abilities into the single label “self-correction.” They are separable, they can fail independently, and each failure implies a different remedy.

Stage	Question	Measured by
B1 — Initiation	Does the continuation contain any verification step at all?	Check Initiation

Stage	Question	Measured by
B2 — Targeting	Does a verification step point into the contamination set?	Audit Hit
B3 — Detection	Does the model state or imply that the target is wrong?	Detection flag
B4 — Repair	Does the final answer come out correct?	Final correctness

Each stage is conditional on the one before it, so the interesting quantities are the transition rates:

- $P(B2 | B1)$ — of the chains that check at all, what fraction check the right place?
- $P(B3 | B2)$ — of the checks aimed correctly, what fraction identify the fault?
- $P(B4 | B3)$ — of the faults identified, what fraction get repaired?

A low $P(B2 | B1)$ is the signature of verification misdirection. A high $P(B2 | B1)$ with low $P(B3 | B2)$ is a model that looks at the right step and fails to see the problem. A high $P(B3 | B2)$ with low $P(B4 | B3)$ is a model that sees the problem and is overruled anyway.

Only B1 and B4 are measurable with existing methods. B2 requires the graph. B3 requires B2, because “detected what?” is meaningless without a target.

7. Hypotheses

H1 — Targeting differs by fault type. (*Primary*)

Plain: Different kinds of damage change not only how often the model checks, but what it checks.

Formal: Audit Hit rate differs significantly across the four perturbation types, and the between-type variation in Audit Hit exceeds the between-type variation in Check Initiation.

Falsification: If Audit Hit is statistically indistinguishable across the four types, H1 fails.

Directional predictions, pre-registered:

Fault type	Check Initiation	Audit Hit	Displacement	Orphan Count
Control	baseline	—	—	baseline
Arithmetic error	raised	high	short	$\approx$ baseline
Authority statement	raised	low	long / undefined	$\approx$ baseline
Skipped step	$\approx$ baseline	not applicable	not applicable	high
Contradictory step	highest	high	short	$\approx$ baseline

H2 — Structure sees what surface cues cannot. (*Primary*)

Plain: Counting phrases like “let me check” tells you a check happened; it cannot tell you the check was aimed at the wrong step.

Formal: Check Initiation and Audit Hit dissociate. $P(\text{Audit Hit} | \text{Check Initiation})$ is substantially below 1 and varies significantly by fault type. A lexical-cue baseline reproduces Check Initiation but fails to reproduce Audit Hit.

Why this is the load-bearing hypothesis: it is a direct, pre-registered contest against an already-published method, using that method’s own measure as the baseline. Both possible

outcomes are publishable. If structure wins, we have shown the graph earns its cost. If the cue baseline matches us, the honest finding is that a simpler instrument suffices for this task — a useful negative result the field does not have.

H3 — Source framing matters independently of content. (*Secondary*)

Plain: Insert the identical wrong number twice — once as the model’s own arithmetic slip, once as a professor’s claim — and the model audits it less often in the professor version.

Formal: Holding the injected numeric value, position, and prefix length constant, Audit Hit is lower under authority framing than under arithmetic framing.

Positioning note: Self-Correction Bench has already established that source framing affects whether correction occurs. Our extension is that it affects *where checking is directed*, which is a different and finer claim. We must state this in the paper rather than let a reviewer state it for us.

H4 — The instrument must be validated on the text it is applied to. (*Methodological*)

Plain: Before trusting any of the above, we check whether our automatic labeller still agrees with humans once the text has been corrupted.

Formal: Labeller–human agreement is computed separately on perturbed and unperturbed subsets. The difference is reported as a statistic. A significant drop gates all automated structural claims.

8. Representation

8.1 Formal pipeline

A chain of thought passes through four stages:

$$C \rightarrow S \rightarrow G \rightarrow M$$

where C is raw text, S the ordered steps, G the typed directed graph, M the metric vector.

- $S = \sigma(C)$, where σ is deterministic segmentation, $S = (s_1 \dots s_n)$
- $\ell = \lambda(S)$, where λ assigns each step one of $\{\text{Compute, Verify, Final}\}$
- $E = E_{\text{seq}} \cup E_{\text{ref}}$, where $E_{\text{seq}} = \{(s_i, s_{i+1})\}$ and $E_{\text{ref}} = \rho(S)$
- $G = (S, \ell, E)$, $M = \mu(G_{\text{clean}}, G_{\text{pert}})$

ρ is heuristic-triggered back-reference resolution. Only λ and ρ involve a model call, and ρ only on flagged steps.

8.2 Nodes: the three-way scheme

Label	Definition	Example
Compute	Asserts a fact, retrieves a premise, or performs a calculation	“ $24 \times 7 = 168$ ”
Verify	Verifies, re-examines, questions, or revises an earlier step	“Let me double-check that”

Label	Definition	Example
Final	States the final answer	“Therefore the answer is B”

Three categories rather than more, because three can be applied consistently by both human annotators and a cheap model. This is a reliability-first choice, not a completeness-first one. The claim is not that three categories exhaust reasoning functions, but that they are the minimal set needed to test H1–H4.

Tie-breaking rule. Some steps fit two categories. A fixed priority applies:

Final > Verify > Compute

Final is positionally unambiguous and should not be diluted. Verify is the rarer, diagnostically load-bearing category, so any step performing genuine backward-looking examination is labelled Verify even if it also computes. Compute is the residual class. This rule appears verbatim in both the annotator instructions and the classifier’s few-shot prompt, so human and model labels are produced under identical rules.

8.3 Edges

Sequence edge (free). Step $i \rightarrow$ step $i+1$ from document order. Guarantees a connected directed acyclic graph for every chain.

Back-reference edge (heuristic-triggered). If a step contains a cue from a fixed list (“as computed above”, “wait, that’s wrong”, “recomputing”, “that contradicts”), a candidate is flagged. A small model resolves which earlier step is referenced, for flagged candidates only, never for every step pair. Direction convention: **later verification node $\rightarrow$ earlier target node**, read as “this check is about that earlier step.”

Independence of cue detection and node labelling. A step containing “wait, that’s wrong” could in principle both receive a Verify label and trigger an edge candidate. The two computations are fully independent: λ sees only the step text and the three-way scheme; ρ runs its cue pass without access to node labels. A step may therefore be Verify with no back-reference edge (checking without a lexical marker), or carry an edge without being Verify (citing an earlier result to use it rather than audit it). This independence is what makes Initiation and Targeting separate measurements rather than one surface cue counted twice. Their empirical correlation is reported descriptively.

8.4 Target categories

Not every verification step has a single identifiable target. Forcing one would manufacture precision we do not have. Four outcomes are permitted:

Code	Meaning	Counts as Initiation?	Counts as Audit Hit?
k	Points at earlier step k	Yes	Yes iff $k \in$ contamination set
NONE	No identifiable earlier target	Yes	No
MULTIPLE	Examines more than one earlier step	Yes	No (primary) / Yes if set intersects contamination (sensitivity)
GLOBAL	Checks the overall solution rather than a step	Yes	No

The primary analysis is strict. A sensitivity analysis treating MULTIPLE-containing-the-injected-node as a hit is reported alongside. Both are specified before data collection.

8.5 Node matching across graphs

Some metrics compare G_{clean} with G_{pert} , which requires knowing which node corresponds to which. This is non-trivial: the skipped-step condition removes a step and shifts every later index; insertions shift indices too.

Nodes are matched primarily by content similarity, with position used only as a tiebreaker — not the reverse. Candidates are scored by normalised edit distance and embedding cosine similarity; the highest-scoring candidate above threshold is the match. Positional proximity breaks ties only when two candidates score within a small margin. Nodes with no above-threshold match are recorded as unmatched.

Position-based matching alone would systematically misreport the skipped-step condition as mass node loss — manufacturing exactly the signature the study seeks. Ablation A3 quantifies the error the naive alternative introduces.

9. Perturbation taxonomy

9.1 Generation mechanism

Prefix-conditioned continuation. A clean chain is generated; a prefix is constructed up to step k ; the fault is applied to that prefix; the model generates the remainder in a **fresh request with no shared conversation history**, so no condition can contaminate another.

9.2 The four faults plus control

Condition	Mechanism	Injected at	Structural prediction
C0 Control	None	—	Reference graph; establishes baseline checking rate
C1 Arithmetic	Modify result of a calculation	A Compute step	Locally correctable — expect high Audit Hit, short displacement
C2 Authority	Replace the same calculation with an attributed claim asserting the same wrong value	Same step as C1	Expect low Audit Hit despite comparable Initiation
C3 Skipped	Omit an intermediate step from the prefix	An intermediate Compute step	Expect high Orphan Count
C4 Contradictory	Insert a step contradicting an earlier step	After a Compute step	Expect high Initiation and high Audit Hit

The C1/C2 matched pair is the tightest experimental control in the design and must be constructed carefully. The naive version inserts an extra authority step while leaving the original calculation intact — but that changes both the numeric content and the chain length, confounding the comparison. Our version replaces the same step in both conditions:

- C1: Step 2: The total is $6 \times 8 = 42$.

- C2: Step 2: A mathematics professor states that $6 \times 8 = 42$.

Identical position, identical false value, identical number of steps. The only difference is attribution. This is a design improvement over the closest prior work, whose corresponding conditions vary on two dimensions at once, and we should say so.

9.3 Contamination set — defined before analysis

For C1, C2 and C4, the injected node is directly known. The contamination set is the injected node plus any later node whose text contains the false value or a quantity arithmetically derived from it. **The propagation rule is fixed at one hop of numeric dependency and written down before any result is inspected**, so that scoring cannot drift once outputs are visible.

For C4, the injected contradictory statement is the primary target; the original contradicted step is recorded as a secondary target and reported in a sensitivity analysis.

9.4 The skipped-step condition needs a ghost node

C3 is structurally different: the damage is a *missing justification*, not a false statement. There is no node to point at, so Audit Hit is undefined by construction.

We handle this with a **ghost node** recorded in metadata only — the deleted step’s text, its original position, and the list of later clean steps that depended on it. C3 is then scored on:

- whether the model reconstructs the missing inference;
- whether it signals uncertainty about the gap;
- **Orphan Count** — later steps relying on the missing fact without re-deriving it.

C3 is excluded from the primary Audit Hit comparison and analysed separately. Stating this openly is better than forcing a metric onto a condition it does not fit.

10. Metrics

Metric	Definition	Funnel stage	Hypothesis
Check Initiation	Binary: does the continuation contain ≥ 1 Verify node?	B1	H1, H2 (control)
Audit Hit	Binary: does any back-reference edge from a Verify node land in the contamination set?	B2	H1, H2, H3
Audit Displacement (graph)	Shortest directed path from the actual target to the injected node; undefined if Audit Hit = 1 or Initiation = 0	B2	H1
Audit Displacement (ordinal)	Absolute difference in step positions, same conditions	B2	Ablation A3
Detection	Binary: does the Verify step state or imply the target is wrong?	B3	H1 (secondary)

Metric	Definition	Funnel stage	Hypothesis
Final correctness	Binary: extracted answer equals gold answer	B4	Practical consequence
Orphan Count	Number of later steps depending on a removed justification without re-deriving it	—	H1 (C3 only)
Continuation mode	Categorical: continued / restarted / rejected prefix / unparsable	—	Data quality

On reporting displacement two ways. The graph version is primary because the whole argument for the graph is that “downstream in the reasoning” differs from “later on the page.” The ordinal version is reported alongside, and **the divergence between the two is itself evidence**: if they agree perfectly, the graph adds nothing on this metric and we say so. Both are pre-specified.

On continuation mode. Models may restart the solution from scratch rather than continue the supplied prefix. A restart may be a legitimate form of recovery, but it is not a within-chain verification back-reference. It is measured and reported as its own category rather than silently discarded.

11. Experiments

#	Tests	Design	Primary statistic
E1	H1	Audit Hit and Displacement across C1, C2, C4 (C3 separate via Orphan Count)	Mixed-effects logistic regression; χ^2 as simple fallback
E2	H2	Verification funnel, and lexical-cue baseline run head-to-head on identical data	$P(B2 B1)$ by condition; agreement/disagreement table
E3	H3	Matched pair C1 vs C2, same problems, same false value	McNemar’s test
E4	H4	Two blind annotators on a stratified damaged/undamaged sample	Cohen’s κ per subset; bootstrap CI on the difference
E5	Secondary	Oracle-location baseline: tell the model which step is damaged, measure repair	Comparison of repair rate with and without location

E5 is cheap and worth including. It connects directly to the established finding that supplying ground-truth error location boosts downstream performance. If our models repair well when told where to look but audit the wrong place when not told, that is a clean demonstration that localization rather than correction capability is the bottleneck — replicating a known result inside our own framework and thereby validating our instrument against an external benchmark.

Ablations

#	Removes	Tests
A1	All back-reference edges	Whether edge detection, the only non-free component, earns its cost
A2	Whole graph, replaced by a regex cue count	Whether the graph is necessary at all (this is E2’s baseline, reported as an ablation too)
A3	Content matching, replaced by index matching	Quantifies the naive alternative’s error, especially on C3

12. Statistical framework

All analyses are specified before data collection, preventing post-hoc metric shopping.

Design is within-problem. The same problem appears in all five conditions, so observations are paired. The primary model is mixed-effects logistic regression with problem as a random intercept:

$$\text{logit } P(\text{AuditHit}) = \beta_0 + \beta_1 \cdot C1 + \beta_2 \cdot C2 + \beta_3 \cdot C4 + u_{\text{problem}}$$

This accounts for some problems being inherently easier and some chains being inherently more check-prone. A simpler clustered logistic regression, or condition-wise proportions with paired contingency tests, is reported as a fallback if convergence fails at our sample size.

Test	When	Library
Mixed-effects logistic regression	E1 primary	<code>statsmodels</code> <code>BinomialBayesMixedGLM</code> / <code>Logit</code> with cluster-robust errors
Chi-square + Cramér’s V	E1 fallback	<code>scipy.stats.chi2_contingency</code> , <code>scipy.stats.contingency.association</code>
Kruskal–Wallis + Dunn’s post-hoc	Displacement across types	<code>scipy.stats.kruskal</code> , <code>scikit_posthocs.posthoc_dunn</code>
Mann–Whitney U	Two-group displacement	<code>scipy.stats.mannwhitneyu</code>
McNemar’s test	E3 matched pair	<code>statsmodels.stats.contingency_tables.mcnemar</code>
Cohen’s κ	E4	<code>sklearn.metrics.cohen_kappa_score</code>
Bootstrap CI	E4 κ difference; all proportions	1,000 resamples, percentile method
Bonferroni correction	Pairwise comparisons	<code>statsmodels.stats.multitest.multipletests</code>

Reporting standard. Effect sizes and confidence intervals are reported alongside every p -value. Conclusions are phrased in effect-size terms — for example, “Audit Hit was 14.2 percentage points lower under authority framing than under an identical arithmetic condition, 95% CI [6.1, 22.5]” — not as bare significance statements.

13. Validation protocol and the gate

This section answers the primary validity risk of the design: that a structural finding could be a parser artifact rather than a real change in reasoning.

13.1 Annotation set

120 chains, stratified: half clean, half damaged, all four fault types proportionally represented, samples from each model. Order shuffled. **Annotators are blind to condition** and are not told which step was injected.

Split into two disjoint parts:

- **Calibration subset (40 chains).** Used to finalise few-shot exemplars, refine the cue list, resolve annotator disagreements. May be inspected freely.
- **Held-out subset (80 chains).** Never inspected during prompt or cue-list development. All reported κ values and the damaged-versus-clean comparison are computed only on this subset.

Without this split, reported agreement would be an optimistic estimate of a pipeline tuned on its own validation data.

13.2 Thresholds

Check	Metric	Threshold
Node labels, human vs human	Cohen’s κ	≥ 0.70
Node labels, model vs human majority	Cohen’s κ	≥ 0.60
Node labels, damaged vs clean	κ per subset independently	No significant drop
Back-reference target, model vs human	Exact-match agreement	≥ 0.70
Segmentation	% boundaries matching human marks	≥ 0.90
Node matching, automated vs human	% agreement on clean-to-damaged mapping	≥ 0.85

The headline statistic is:

$$\Delta_{\text{reliability}} = \text{Agreement}(\text{clean}) - \text{Agreement}(\text{damaged})$$

reported with a bootstrap 95% confidence interval. If the interval excludes zero, the drop is real and is disclosed rather than averaged away.

13.3 The gate

Outcome	Action
$\kappa(\text{damaged}) \geq 0.60$ and Δ CI includes 0	Proceed to full-scale automated labelling
$\kappa(\text{damaged}) \geq 0.60$ but Δ CI excludes 0	Proceed, report Δ prominently, re-derive every structural claim with the drop stated
$\kappa(\text{damaged}) < 0.60$	Do not report automated structural findings. Use human labels on the smaller analysis-eligible subset, report the reduced-power analysis honestly, and report the parsing limitation as a methodological finding in its own right

This gate is not advisory. The execution plan sequences the gate decision before full-scale generation and labelling are committed to. The third branch is a legitimate outcome, not a failure: given that comparable designs validate only on clean traces, a documented demonstration that automated structural parsers are unreliable under corrupted input is a real contribution.

14. Expected outcomes

All three are scientifically meaningful. The project does not depend on a predetermined positive result.

Outcome A — H1 supported. Audit Hit differs significantly and in the predicted directions: authority framing suppresses correct targeting while leaving checking language intact; contradictions raise both; arithmetic errors are locally corrected; skipped steps raise Orphan Count. This establishes that fault types leave distinguishable structural signatures and that verification targeting is a usable diagnostic channel.

Outcome B — signatures exist but are not type-specific. Targeting degrades under damage relative to control, but the fault types are not reliably separable. Still informative: checking is measurably misdirected, but is a sensitivity indicator rather than a diagnostic one. This would redirect the programme toward finer-grained taxonomies rather than abandoning the instrument.

Outcome C — H1 rejected. Targeting is indistinguishable across types and from control. A valuable negative result the field lacks: graph structure over chain-of-thought would not carry the diagnostic information several recent frameworks implicitly assume. Combined with the E4 validation statistics — which distinguish “no signal exists” from “our parser cannot detect it” — this outcome is interpretable rather than merely null. That distinction is exactly what the gate buys.

Independently of A/B/C, two results are near-certain to be reportable:

1. The verification funnel measured for the first time, giving $P(B2 \mid B1)$ — the fraction of checks that are correctly aimed. Whatever that number is, it is new.
 2. The damaged-versus-clean annotation agreement difference, which no comparable pipeline reports.
-

15. Contributions

1. **The verification funnel** — a four-stage decomposition (initiation, targeting, detection, repair) separating abilities that existing work collapses into “self-correction,” with a measurement procedure for each stage.
2. **Audit Hit and Audit Displacement** — metrics for whether spontaneous verification is structurally directed at a known fault, computable only from a graph representation.
3. **A validated labelling protocol** with agreement reported separately for damaged and undamaged text and enforced as a reporting gate — addressing a validity gap present in every comparable prior design.
4. **A tightly matched framing contrast** (C1 vs C2) holding numeric content, position and chain length constant, isolating source attribution from content.
5. **A pre-registered statistical framework** — metrics, tests, effect-size reporting and multiple-comparison correction fixed before data collection.
6. **A cost-bounded pipeline** usable without frontier-model budgets, with a stated cost envelope rather than an unquantified claim of cheapness.

Contributions 1, 3, 5 and 6 are methodological and stand regardless of whether H1 is supported.

16. Limitations

- **Single primary domain.** GSM8K is arithmetic. Generalisation to non-arithmetic reasoning is exploratory only.
- **Short chains bound the effect.** GSM8K chains are typically 4–8 steps, which limits how far a misdirected check can land from the fault. If Audit Hit shows no variance in the pilot, a longer-chain dataset (MATH-500) is the indicated remedy.
- **Reliability-first taxonomy.** Three categories will not capture every reasoning function; planning, hedging and restatement are absorbed into Compute.
- **Surface-cue dependence.** Back-references without lexical markers are missed. This is a recall limitation and is disclosed with measured agreement rather than hidden. We additionally report the **cue-blind verification rate** — the fraction of human-labelled Verify steps containing no cue from our list — which bounds the ceiling of every cue-based method including our own.
- **Fixed injection position.** Post-first-computation-step trades ecological validity for control. Position-varying faults are future work.
- **Visible reasoning is not internal reasoning.** We measure the structural direction of verification *expressed in generated text*, not the model’s unobservable internal computation. Chain-of-thought text can be post-hoc, incomplete, or strategically formatted. All claims are phrased accordingly.
- **Edges are discourse relations, not proven dependencies.** Recent work reports that mechanistically measured causal step dependencies do not align with language-level discourse relations. A back-reference edge records what the text says it is about, which is the object of study, not a claim about internal causation.
- **Signature $\neq$ mechanism.** Even under Outcome A we establish correlation between fault type and verification targeting, not a causal account of internal computation.

17. Threats to validity and their controls

Threat	Control
Structural change reflects parser behaviour on unusual text, not reasoning change	Human validation on the damaged subset; κ per subset; hard gate (§13.3)
Index shifts from insertion or removal manufacture apparent node loss	Content-first matching with position as tiebreaker; ablation A3 quantifies the naive alternative’s error
One lexical cue counted twice, as node label and as edge	Node classification and cue detection computed independently; correlation reported
Structural signal is a verification-phrase count in disguise	E2 / ablation A2: cue-count baseline reported head-to-head
Baseline incompetence, not the fault, causes failure	Items filtered to base-model-correct before injection
Floor effect: base model rarely checks at all	Baseline Check Initiation measured in the pilot audit; explicit go/no-go
C1 vs C2 differ in more than framing	Matched replacement design: identical value, position, and chain length
Scoring drifts once outputs are visible	Contamination-set propagation rule fixed in writing before analysis
Post-hoc metric shopping across metrics $\times$ types	All metrics, tests and thresholds pre-registered
Multiple-comparison inflation	Bonferroni correction on pairwise tests
Prompt tuned on the data used to validate it	Calibration / held-out split of the annotation set
Annotators biased toward the hypothesis	Annotators blind to condition and to injection location

Threat	Control
Cross-condition contamination	Independent API request per condition; no shared conversation
Restarts silently counted as non-verification	Continuation mode measured and reported as its own category

18. Scope: this semester versus the full thesis

The Design Project is one slice of a longer research programme. The boundary is stated here so it is not blurred later.

Achievable this semester

Weeks	Phase	Status
3	Setup: repository, prompts, model access, dataset filtering	Full
4–5	Pilot (30 problems $\times$ 5 conditions) and dataset audit	Full
5–7	Human validation and gate decision (E4)	Full — this is the headline
8–10	Full generation, one model family , 200–300 problems $\times$ 5 conditions	Reduced scope
10–11	E1, E2, E3	Full
11–12	Ablations A1, A3; E5 if time; build the prototype tool	Partial
13–14	Report, presentation, artifact submission	Full

Explicitly deferred to the thesis

1. **Second model family and cross-model replication.** The closest prior work ran thirteen models; we cannot compete on that axis, it doubles cost, and it introduces normalisation problems across models of differing verbosity.
2. **Ablation A2 in its strong form** — replicating the published doubt-classifier baseline exactly, rather than a simple regex count.
3. **Longer-chain datasets** (MATH-500) to give misdirection room to appear.
4. **Position-varying injection.**
5. **Detection (B3) as a primary result.** It is measured and reported descriptively this semester, but establishing it properly requires its own annotation protocol.
6. **Publication-grade writeup.**

Prototype deliverable

The course requires a working tool alongside the report. Minimum honest scope:

- User selects a GSM8K problem and one of four faults.
- Tool generates the clean chain, applies the fault, generates the continuation.
- Renders both graphs side by side: nodes coloured by label, back-reference edges dashed, the injected node outlined, unmatched nodes greyed.

- Prints the funnel panel: Initiation, Audit Hit, Displacement, Orphan Count, final answer, and a one-line diagnosis.
- A second view shows aggregate E1 results over the full run.

All four faults must be real, both graphs must be built by the actual pipeline, and all metrics must be computed by the same functions used in the analysis. No hardcoded example. Generations are cached to disk so the demonstration does not depend on a live network connection.

19. Budget

Cost driver	Volume	Estimate
Clean generation	~500 chains $\times$ ~300 tokens	\$2 – \$8
Continuation generation	~1,500 chains $\times$ ~300 tokens	\$5 – \$15
Node labelling	~12,000 step classifications, batched	\$3 – \$12
Back-reference resolution	~10–20% of steps flagged $\rightarrow$ ~1,500–2,500 calls	\$1 – \$6
Node-matching embeddings	~12,000 local embeddings	\$0
Dataset	GSM8K, freely available	\$0
Human annotation	120 chains $\times$ 2 annotators $\approx$ 8–10 person-hours, team members	\$0
Total		\$11 – \$41

Self-hosted on a university GPU, compute cost falls to zero. No frontier-model dependency exists anywhere in the pipeline.

20. Final thesis statement

When a language model’s reasoning is disturbed and its answer changes, the field currently records that it failed. Yet the ways reasoning can break are not interchangeable. A model that never checks its work is failing differently from one that checks the wrong step, which is failing differently again from one that checks the right step and is overruled by a false authority. This project proposes that these differences are measurable, and that the measurement requires structure: specifically, whether a verification step is connected by a back-reference to the step that was damaged. By representing each chain of thought as a small typed directed graph built almost entirely from cheap deterministic operations, by planting the fault ourselves so that its location is known without annotation cost, by matching graphs before and after damage on content rather than position, and by validating the parsing pipeline against human annotation on damaged text specifically under a gate that blocks reporting if validation fails, this work aims to determine whether a model’s spontaneous self-checking is aimed at anything at all — and thus whether the visible appearance of verification can be trusted as evidence that verification occurred.

21. References

1. J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, D. Zhou. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS, 2022. arXiv:2201.11903.
2. K. Cobbe, V. Kosaraju, M. Bavarian, et al. *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168, 2021. (GSM8K)
3. A. V. Aravindan, M. Kejriwal. *Fragile Thoughts: How Large Language Models Handle Chain-of-Thought Perturbations*. arXiv:2603.03332, 2026.
4. A. von Recum, L. Gierbach, Z. Akata. *Are Reasoning LLMs Robust to Interventions on their Chain-of-Thought?* arXiv:2602.07470, 2026.
5. K. Tsui. *Self-Correction Bench: Uncovering and Addressing the Self-Correction Blind Spot in Large Language Models*. arXiv:2507.02778, 2025.
6. J. Lee, S. Mukherjee, D. Hakkani-Tür, J. Hockenmaier. *ReasoningFlow: Semantic Structure of Complex Reasoning Traces*. arXiv:2506.02532, 2025. Extended as *ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces*, arXiv:2606.05402, 2026.
7. G. Tyen, H. Mansoor, V. Cărbune, P. Chen, T. Mak. *LLMs Cannot Find Reasoning Errors, but Can Correct Them Given the Error Location*. Findings of ACL 2024, pages 13894–13908. arXiv:2311.08516.
8. Z. Xiong, et al. *Mapping the Minds of LLMs: A Graph-Based Analysis of Reasoning LLM*. arXiv:2505.13890, 2025.
9. C. Zheng, et al. *ProcessBench: Identifying Process Errors in Mathematical Reasoning*. arXiv:2412.06559, 2024.
10. H. Lightman, V. Kosaraju, Y. Burda, et al. *Let’s Verify Step by Step*. ICLR 2024. arXiv:2305.20050.
11. Z. Ling, Y. Fang, X. Li, Z. Huang, M. Lee, R. Memisevic, H. Su. *Deductive Verification of Chain-of-Thought Reasoning*. NeurIPS 2023. arXiv:2306.03872.
12. T. Lanham, et al. *Measuring Faithfulness in Chain-of-Thought Reasoning*. arXiv:2307.13702, 2023.
13. M. Turpin, J. Michael, E. Perez, S. R. Bowman. *Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting*. NeurIPS 2023. arXiv:2305.04388.
14. M. Besta, N. Blach, A. Kubicek, et al. *Graph of Thoughts: Solving Elaborate Problems with Large Language Models*. AAAI 2024. arXiv:2308.09687.
15. P. C. Bogdan, U. Macar, N. Nanda, A. Conmy. *Thought Anchors: Which LLM Reasoning Steps Matter?* NeurIPS 2025. arXiv:2506.19143.
16. Z. Zhao, Y. Koishekenov, X. Yang, N. Murray, N. Cancedda. *Verifying Chain-of-Thought Reasoning via Its Computational Graph*. arXiv:2510.09312, 2026.
17. *Structure Enables Effective Self-Localization of Errors in LLMs*. arXiv:2602.02416, 2026.

18. J. Cohen. *A Coefficient of Agreement for Nominal Scales*. Educational and Psychological Measurement, 20(1):37–46, 1960.
19. O. J. Dunn. *Multiple Comparisons Using Rank Sums*. Technometrics, 6(3):241–252, 1964.
20. Q. McNemar. *Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages*. Psychometrika, 12(2):153–157, 1947.

Priority reading list

The five papers to read in full, in this order:

1. **Fragile Thoughts** (ref. 3) — closest experimental setup; establishes what we must not claim.
2. **von Recum et al.** (ref. 4) — closest dependent variable; establishes the measurement we must exceed.
3. **Self-Correction Bench** (ref. 5) — the matched-framing precedent for H3; source of the blind-spot finding.
4. **Tyen et al.** (ref. 7) — establishes that localization, not correction, is the bottleneck; motivates E5.
5. **ReasoningFlow** (ref. 6) — the graph parser; its clean-text-only validation is the gap H4 occupies.

Refs. 8, 9, 10, 11 and 12 are read at abstract-and-method level and cited without full study.